SGAE-Data-Lake: Review, README Draft, and Architecture Plan

Scope and evidence

Reviewed the public repository landing page and the root listing of its main branch on October 12, 2026.

Sources:

- **Repository**
- **Main branch listing**

The retrieved root listing showed **README.md**. Its text was the project title and "SISTEM MANAGEMENT ALL WEBSITE OPERATIONS." No application code, deployment configuration, or tests were exposed in that retrieved listing. This is a limited public-page review, not an executed code audit or proof about other branches, private services, or deployments.

All capabilities and components below are proposals, not verified existing features. Working assumption: the project should collect website operational data and support centralized oversight. Cloud provider, budget, traffic, data residency, and source systems remain undecided.

1. Project review

Current assessment

The available documentation establishes a project name and a broad website-operations goal, but does not demonstrate an implemented data lake or management application. Treat the repository as an early project scaffold until implementation evidence is available.

Area	Evidence from retrieved pages	Recommended next step
Purpose	Short website-operations description	Define users, source systems, and first use case
Implementation	No code exposed in retrieved root listing	Add a small ingestion service and sample input
Setup	No instructions in retrieved README	Document prerequisites, configuration, and tested startup steps
Data model	No schema described	Define event format and data retention
Security	No security documentation exposed	Define authentication, access boundaries, and secret handling
Testing	No tests exposed	Add validation, duplicate, isolation, and failure tests
Deployment	No deployment instructions exposed	Document one repeatable development environment
Licensing	No license terms exposed	Choose an appropriate license before inviting reuse

Priorities

First: Replace the short description with honest documentation that distinguishes planned and implemented functionality.

Next: Build one end-to-end path: ingest a synthetic website event, validate it, retain the raw input, write a normalized record, and retrieve a daily summary.

Then: Add authentication, website-level authorization, monitoring, recovery tests, and a documented deployment process.

Boundaries

A data lake stores and processes operational data. It should not directly execute arbitrary website administration commands. Keep data collection separate from privileged actions such as deployment, deletion, DNS changes, or credential rotation. Any future action service should require scoped permissions, explicit approvals where appropriate, and audit records.

2. Improved README draft

The section below is replacement README content. It intentionally avoids installation commands that cannot yet be tested.

SGAE-Data-Lake

A proposed central platform for collecting, organizing, and analyzing website operational data.

Project status: planning / initial documentation. The publicly reviewed main-branch listing currently exposes a brief README; the features described here are a proposed roadmap, not verified working functionality.

Purpose

SGAE-Data-Lake aims to give website operators a shared view of operational events, ingestion health, and historical trends while keeping data collection separate from privileged website-management actions.

Planned capabilities

- Collect authorized website events and operational logs.
- Preserve raw inputs for controlled replay and troubleshooting.
- Validate schemas and quarantine malformed records.
- Normalize data for reporting and analysis.
- Provide authenticated, website-scoped access to summaries.
- Track ingestion failures and data freshness.

Initial use case

Use synthetic website availability events to demonstrate a complete ingestion-to-summary workflow before connecting production systems.

Proposed data flow

Website sources → authenticated ingestion → raw storage → validation and normalization → curated datasets → read-only reporting API or dashboard.

Implementation status

- Project name and high-level description: documented.
- Ingestion service, storage, transformations, dashboard, tests, and deployment: proposed; implementation not verified in the public-page review.

Getting started

A tested runnable setup is not yet documented. Before publishing startup commands:

1. Select the implementation stack and supported runtime versions.
2. Add dependency manifests and a development configuration example.
3. Implement a synthetic-data ingestion path.
4. Test setup from a clean environment.
5. Add the verified commands and expected results here.

Proposed repository organization

- `services/ingestion/`: source authentication and event intake.
- `services/api/`: authorized access to summaries.
- `pipelines/`: validation and transformation jobs.
- `schemas/`: versioned event definitions.
- `tests/`: unit, integration, authorization, and recovery tests.
- `infra/`: deployment configuration.
- `docs/`: architecture, data governance, and operational guides.

- `.env.example`: configuration names and safe placeholders only.

This is a suggested structure, not a description of existing directories.

Security and data handling

- Never commit credentials, private keys, access tokens, or real customer datasets.
- Use synthetic examples during development.
- Minimize personal data and define retention before production ingestion.
- Apply least-privilege access and website-level authorization.
- Use encrypted transport and appropriate storage encryption.
- Keep administrative operations outside the analytics ingestion path.
- Document a private security-reporting channel before production use.

Roadmap

- Define one operational event schema.
- Build a local synthetic-data ingestion prototype.
- Add validation, deduplication, and summary queries.
- Add authentication and authorization tests.
- Add deployment documentation and recovery checks.
- Expand to additional sources only after the first path is reliable.

Contributing

For proposed changes, explain the problem, intended behavior, tests, and any security or data-handling implications. Contribution and review policies should be finalized by the maintainer.

License

License selection is pending confirmation. Do not assume permission to reuse or redistribute the project without reviewing applicable license terms.

3. Proposed data-lake architecture

Design approach

Begin with a small, cloud-neutral prototype. Adopt managed services when workload, budget, and residency requirements justify them. Do not start with a large streaming cluster or multi-region deployment without a demonstrated need.

Components

Component	Responsibility	Suggested prototype choice
Sources	Emit authorized operational events	Synthetic JSON events, then one website connector
Ingestion service	Authenticate sources, validate envelopes, enforce limits	Small HTTP service in the team's preferred language
Raw storage	Preserve received payloads and ingestion metadata	Private object storage
Processing job	Validate, normalize, deduplicate, and quarantine	Scheduled worker
Curated storage	Store analysis-ready datasets	Partitioned columnar files when appropriate
Operational database	Track websites, sources, job status, and permissions	Relational database
Reporting API	Serve authorized summaries	Read-only API initially
Operator view	Show freshness, failures, and daily summaries	Minimal dashboard after ingestion works
Monitoring	Track errors, lag, retries, and storage usage	Structured logs and basic alerts

These are design suggestions, not claims about the repository's current stack.

Data organization

Raw zone: original payload, source identity, receipt time, and schema version. Restrict access and apply retention; do not retain sensitive data indefinitely for replay.

Validated zone: typed records with normalized timestamps and identifiers. Invalid records go to a restricted quarantine location with an error reason.

Curated zone: website-level daily summaries and other datasets with documented definitions. Separate summary access from access to raw logs.

Partition by event date and, where justified by scale and access design, website. Avoid excessively small files and unnecessary partitions in the initial prototype.

Proposed event contract

Each operational event should include:

- Unique event identifier for deduplication.
- Authorized website identifier.
- Source identifier.
- Event type, such as availability check or deployment status.
- Event occurrence time in UTC.
- Schema version.
- Event-specific payload with explicit allowed fields.

Add receipt time on the server. Validate timestamp bounds, payload size, supported event types, and the source's authority to report for the website. Avoid raw IP addresses, access tokens, and personal data unless a defined use case and lawful handling requirements support collection.

Reliability controls

- Accept success only after durable capture, or clearly distinguish queued acceptance from completed processing.
- Use idempotent handling so repeated submissions do not inflate reports.
- Retry transient failures with limits and backoff.
- Quarantine permanently invalid records without blocking valid data.

- Support replay through the same validation rules.
- Define how late events affect already-produced summaries.
- Track ingestion lag, invalid-record rate, job failures, and last successful source update.
- Test backup restoration and document recovery expectations.

Security controls

- Separate source credentials from operator accounts.
- Enforce website or tenant isolation in every access path, including storage and exports.
- Keep raw storage private and prohibit public bucket access.
- Store secrets outside source control; rotate them when exposed.
- Separate development and production data and credentials.
- Use least-privilege service identities and audit sensitive access.
- Define deletion, retention, and access-review processes before production.
- Treat hashes as integrity checks, not proof of identity, certification, or external endorsement.

MVP delivery sequence and acceptance criteria

Stage A: scope and contract

Agree on one website source, one event type, access roles, retention, and a target daily summary. Publish a sample event schema and synthetic examples.

Done when: the team can explain which data is collected, why, who may access it, and when it is deleted.

Stage B: ingestion and storage

Implement authenticated intake, durable raw capture, schema validation, deduplication, and quarantine.

Done when: a valid event reaches validated storage; malformed events are rejected or quarantined; duplicate events do not create duplicate analytical records.

Stage C: query and reporting

Produce a daily summary and expose it through a website-scoped API or minimal dashboard.

Done when: sample results match known synthetic inputs and an operator cannot access another website's restricted data.

Stage D: operational readiness

Add monitored jobs, bounded retries, deployment instructions, automated checks, and restore tests.

Done when: a clean environment can reproduce the workflow, a failed job can recover without data inflation, and the team can demonstrate restoration of required data.

Decisions still required before production

- Which websites and systems supply data?
- Is this analytics-only, or does a separate management-action service need to be designed?
- What are the expected event volume and freshness requirements?
- Which cloud or hosting environment is approved?
- What budget and operating responsibilities apply?
- Are multiple organizations sharing the platform?
- What residency, privacy, retention, and recovery requirements apply?

Recommended first milestone

Deliver one tested synthetic-event ingestion-to-daily-summary workflow. Keep all unimplemented functionality labeled as planned. This creates implementation evidence before expanding the platform's scope.